EXP NO: 3	*CLASSIFICATION WITH DECISION TREES*

AIM:

To build a classification model using a Decision Tree classifier, train it on the Iris dataset, and evaluate it using accuracy, confusion matrix, classification report and feature importance.

ALGORITHM:

1. Import the required libraries (pandas, seaborn, matplotlib, scikit-learn).
2. Load the Iris dataset and explore it (shape, missing values, class distribution).
3. Separate the features (X) and the target (y).
4. Split the dataset into training and testing sets using train_test_split().
5. Create the model using DecisionTreeClassifier() with the Gini criterion and

   max_depth=3.
6. Train the model using fit() and predict the test data using predict().
7. Evaluate the model using accuracy, confusion matrix and classification report.
8. Find the feature importance and visualise the tree using plot_tree().

CODE:

```
# 1. Import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix,

    classification_report

# 2. Load and explore the dataset
iris = load_iris(as_frame=True)
df = iris.frame
pd.set_option("display.max_columns", None)
print(df.head())
print("\nShape:", df.shape)
print("\nMissing values:\n", df.isnull().sum())
print("\nClass distribution:\n", df['target'].value_counts())


# 3. Define features and target, then split
```

```
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# 4. Build and train the Decision Tree
dt_model = DecisionTreeClassifier(criterion='gini', max_depth=3,
    random_state=42)
dt_model.fit(X_train, y_train)

# 5. Predict and evaluate
y_pred = dt_model.predict(X_test)
print("\nDecision Tree Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=iris.target_names))

# 6. Feature importance
importance = pd.Series(dt_model.feature_importances_, index=iris.feature_names)
print("Feature Importance:\n", importance.sort_values(ascending=False))

# 7. Visualise the tree
plt.figure(figsize=(14, 8))
plot_tree(dt_model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True, rounded=True)
plt.title("Decision Tree (max_depth=3)")
plt.show()

# 8. Confusion matrix heatmap and feature importance plot
plt.figure(figsize=(11, 4))
plt.subplot(1, 2, 1)
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
    cmap='Oranges',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title("Confusion Matrix"); plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.subplot(1, 2, 2)
importance.sort_values().plot(kind='barh', color='teal')
plt.title("Feature Importance")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                 5.1               3.5                1.4               0.2
1                 4.9               3.0                1.4               0.2
2                 4.7               3.2                1.3               0.2
3                 4.6               3.1                1.5               0.2
4                 5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0

Shape: (150, 5)

Missing values:
 sepal length(cm)     0
sepal width (cm)      0
petal length (cm)     0
petal width (cm)      0
target                0
dtype: int64

Class distribution:
target
0    50
1    50
2    50
Name: count, dtype: int64

Decision Tree Accuracy: 0.9777777777777777

Confusion Matrix:
 [[15  0 0]
 [ 0 14  1]
 [ 0  0 15]]

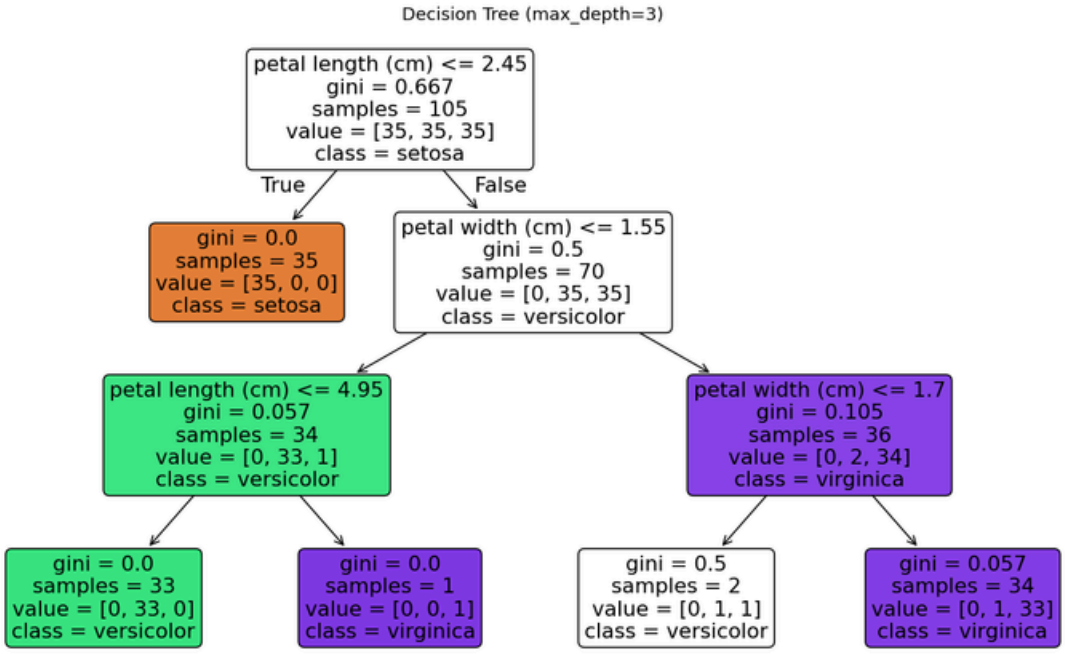
Classification Report:
               precision    recall  f1-score   support

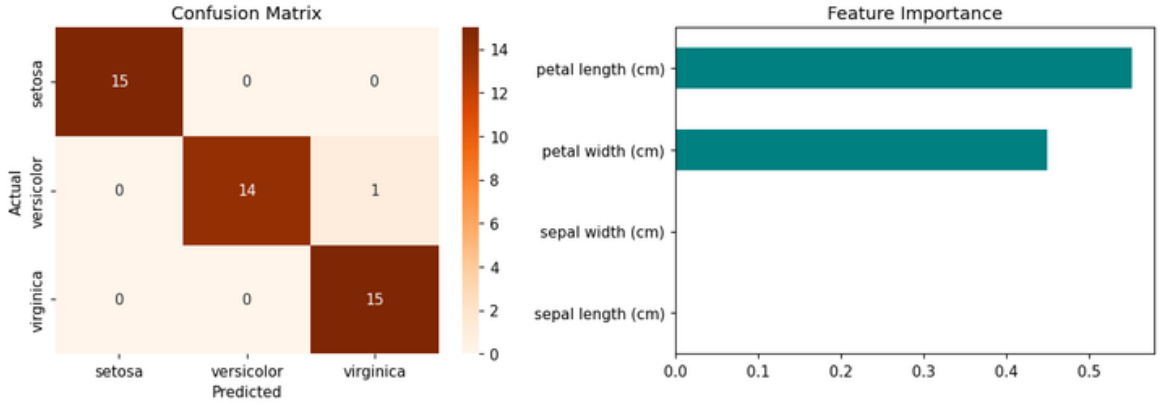
      setosa       1.00      1.00      1.00        15
  versicolor       1.00      0.93      0.97        15
   virginica       0.94      1.00      0.97        15

    accuracy                           0.98        45
macro    avg       0.98      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45

Feature Importance:
petal  length  (cm)   0.550877
petal   width  (cm)  0.449123
sepal width (cm)      0.000000
```

```
sepal length (cm)     0.000000
dtype: float64
```





RESULT:

The Decision Tree classifier was successfully implemented on the Iris dataset. The model achieved an accuracy of 97.78% on the test data, with only one versicolor sample misclassified as virginica. Petal length and petal width were found to be the most important features for classification.